

Embedded Systems & IoT Internship – Task 1

Submitted By

B. Meghana

Role

Junior Embedded Engineer

Department

Electronics and Communication Engineering

Internship Program

Maincrafts Technology

College Name

Stanley College Of Engineering And Technology For Women

Date

18-05-2026

Introduction to Embedded Systems

An embedded system is a combination of computer hardware and software designed to perform a dedicated function within a larger electronic system. Unlike general-purpose computers, embedded systems are developed for specific tasks and operate with real-time constraints, reliability, and efficiency. These systems are integrated into everyday devices such as home appliances, automobiles, industrial automation equipment, healthcare devices, communication systems, and smart electronic products.

Embedded systems consist of microcontrollers or microprocessors, memory units, sensors, actuators, communication interfaces, and software programs. The hardware and software work together to collect data from the environment, process it, and perform automated actions based on predefined instructions.

The rapid growth of the Internet of Things (IoT) has significantly increased the importance of embedded systems in modern technology. IoT devices rely on embedded systems for sensing, communication, monitoring, and automation. Smart homes, wearable devices, autonomous vehicles, and industrial monitoring systems are examples of applications that utilize embedded systems extensively.

Embedded systems are preferred because of their:

- Low power consumption
- Compact size
- Real-time operation

Real-World Applications of Embedded Systems

Embedded systems are integrated into various real-world applications where automation, monitoring, and intelligent control are required. Some important applications include:

Smart Home Automation

Embedded systems are used in smart lighting systems, automatic door control, smart fans, air conditioning systems, and security monitoring systems. Sensors and controllers help automate household appliances and improve energy efficiency.

Automotive Systems

Modern vehicles contain multiple embedded systems for engine control, braking systems, airbags, infotainment systems, and navigation systems. Embedded controllers ensure vehicle safety and performance.

Healthcare Devices

Medical equipment such as heart rate monitors, glucose monitoring systems, digital thermometers, ventilators, and wearable health devices use embedded systems for accurate monitoring and diagnostics.

Industrial Automation

Factories use embedded systems for process control, robotic automation, temperature monitoring, and machine operation. These systems improve productivity and reduce human error.

Difference Between Microcontroller and Microprocessor

Microcontroller	Microprocessor
A microcontroller is designed for dedicated embedded applications.	A microprocessor is designed for general-purpose computing applications.
It contains CPU, RAM, ROM, and input/output ports integrated into a single chip	It mainly contains only the CPU and requires external memory and peripherals.
It consumes low power and is suitable for battery-operated devices.	It consumes more power due to high processing capability.
Used in embedded systems such as Arduino boards, automation systems, and appliances.	Used in personal computers, laptops, and servers.
Suitable for real-time applications and hardware interfacing.	Suitable for multitasking and complex processing tasks.
Example: ATmega328P microcontroller in Arduino UNO.	Example: Intel Core i5 processor.

Component Study

➤ Arduino UNO

Arduino UNO is one of the most popular open-source microcontroller development boards used for embedded systems and IoT applications. It is based on the ATmega328P microcontroller and provides a user-friendly platform for prototyping electronic projects.

Features of Arduino UNO

- ATmega328P microcontroller
- USB programming support
- Easy integration with sensors and actuators
- Supports Embedded C and Arduino programming

Applications of Arduino UNO

- Sensor interfacing applications
- Smart monitoring systems

Role in This Project

In the Smart Lighting System, Arduino UNO acts as the main controller. It receives analog data from the LDR sensor, processes the sensor values, and controls the LED automatically.

➤ ESP32

ESP32 is an advanced microcontroller module widely used in IoT applications because of its built-in Wi-Fi and Bluetooth communication capabilities. It is developed by Espressif Systems and supports low-power wireless communication.

Features of ESP32

- Dual-core processor
- Built-in Wi-Fi and Bluetooth
- High processing speed
- Low power operation

Applications of ESP32

- Smart IoT devices
- Wireless sensor networks

➤ **Raspberry Pi Pico**

Raspberry Pi Pico is a low-cost microcontroller development board based on the RP2040 microcontroller chip. It is designed for embedded applications, automation projects, and educational electronics.

Features of Raspberry Pi Pico

- RP2040 dual-core microcontroller
- Programmable using C and MicroPython
- Multiple GPIO pins
- Low power consumption

Applications of Raspberry Pi Pico

- Embedded system development
- Sensor interfacing
- Automation systems

➤ **Temperature Sensor**

A temperature sensor is an electronic device used to measure the degree of heat present in an environment or object and convert it into an electrical signal.

One commonly used sensor in basic electronics projects is the **LM35 temperature sensor**. It provides an analog output voltage that is directly proportional to temperature in Celsius. For every 1°C rise in temperature, the output increases by 10 mV.

Features:

- Operates in a wide temperature range (typically -55°C to 150°C)
- Linear output (easy to interface with microcontrollers like Arduino)
- Low self-heating

Applications:

- Weather monitoring systems
- Home automation
- Industrial temperature control systems

➤ **Motion Sensor**

A motion sensor detects physical movement within a given area and converts it into an electrical signal. The most commonly used type is the **PIR (Passive Infrared) sensor**.

A PIR sensor detects infrared radiation emitted by human bodies or warm objects. When motion is detected, the sensor outputs a HIGH signal.

Features:

- Detects infrared radiation changes
- Adjustable sensitivity and delay time
- Low power consumption

Applications:

- Security alarm systems
- Automatic lighting systems

- Smart home devices

➤ 3. Light Sensor

A light sensor is used to measure the intensity of light in the surrounding environment. The most commonly used light sensor in basic projects is the **LDR (Light Dependent Resistor)**.

The resistance of an LDR decreases when light intensity increases and increases when light intensity decreases. This change in resistance is used to control circuits automatically.

Features:

- Simple and low cost
- Responds to varying light levels
- Easy to interface with Arduino

Applications:

- Automatic street lights
- Brightness control systems
- Solar tracking systems

Mini Project – Smart Lighting System Using Arduino and LDR

Objective

The objective of this project is to design and simulate a Smart Lighting System using Arduino UNO and an LDR sensor in Tinkercad. The system automatically controls lighting based on surrounding environmental light intensity.

The project demonstrates the concepts of:

- Sensor interfacing
- Embedded programming
- Automation
- Analog signal processing
- Smart energy-efficient systems

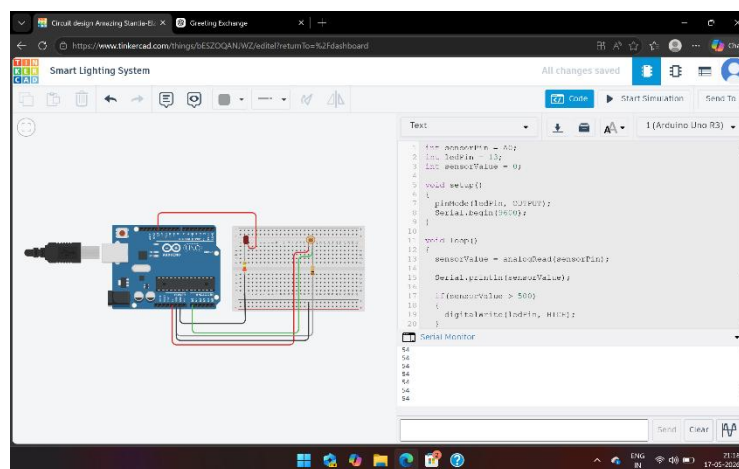
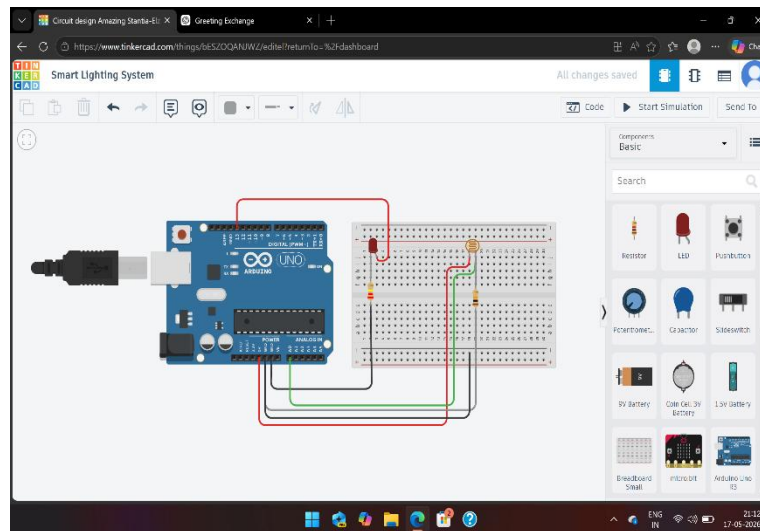
Components Used

The following components were used for the implementation of the Smart Lighting System:

- Arduino UNO
- Breadboard
- LDR Sensor
- LED
- 220Ω Resistor
- 10kΩ Resistor
- Jumper Wires
- Tinkercad Simulation Platform

Circuit Diagram

The circuit consists of an LDR sensor connected to the analog input pin A0 of Arduino UNO through a voltage divider configuration. The LED is connected to a digital output pin through a current-limiting resistor.



Arduino Code

```
// LDR sensor connected to analog pin A0
int sensorPin = A0;
```

```
// LED connected to digital pin 13
int ledPin = 13;
```

```
// Variable to store sensor readings
int sensorValue = 0;
```

```
void setup()
{
  // Configure LED pin as output
  pinMode(ledPin, OUTPUT);
```

```
// Start serial communication
Serial.begin(9600);
}
```

```
void loop()
```

```

{
// Read analog value from LDR sensor
sensorValue = analogRead(sensorPin);

// Display sensor value in Serial Monitor
Serial.println(sensorValue);

// If environment is dark
if(sensorValue > 500)
{
// Turn ON LED
digitalWrite(ledPin, HIGH);
}
else
{
// Turn OFF LED in bright light
digitalWrite(ledPin, LOW);
}

// Small delay for stability
delay(100);
}

```

Code Explanation

The Arduino program continuously reads analog values from the LDR sensor through analog pin A0. The sensor values are displayed in the Serial Monitor for observation and analysis.

A threshold value of 500 is used to determine environmental lighting conditions. When the sensor value becomes greater than 500, the Arduino interprets it as a dark condition and turns the LED ON. When the sensor value is lower than 500, the environment is considered bright and the LED remains OFF.

The delay function ensures stable operation of the system and prevents unnecessary rapid switching.

Working Principle

The Smart Lighting System works based on the intensity of surrounding light. The LDR sensor continuously monitors environmental light conditions and produces varying analog voltage values.

The Arduino UNO reads these analog values through analog pin A0 using the analog-to-digital conversion process. Based on the programmed threshold condition, Arduino controls the LED automatically.

System Operation

- In bright conditions, the sensor value remains low and the LED stays OFF.
- In dark conditions, the sensor value increases and the LED turns ON automatically.

This process demonstrates automatic light control using embedded systems and sensor interfacing.

Observation and Results

The Smart Lighting System was successfully simulated in Tinkercad.

Observed Sensor Values

- Bright condition: approximately 54
- Dark condition: approximately 974

Output Observation

- LED turned OFF in bright light.
- LED turned ON in dark conditions.

Applications of the Project

- Automatic street lighting systems
- Smart home automation
- Energy-efficient lighting systems
- Garden lighting systems

Advantages of the System

- Automatic operation
- Low power consumption
- Simple and cost-effective design
- Easy implementation

Conclusion

The Smart Lighting System using Arduino UNO and LDR sensor was successfully designed and simulated using the Tinkercad platform. The project demonstrated the practical implementation of embedded systems concepts including sensor interfacing, analog data processing, and automation.

The system automatically controlled the LED based on surrounding light intensity, thereby demonstrating an energy-efficient smart lighting solution. This project also enhanced understanding of Arduino programming, circuit design, and IoT-based automation systems.